

EECS 3311 Software Design

Summer 2026

Section E

Deliverable 1 (Total Marks 100)

Overview

In this project, you will learn to design software from scratch, i.e., analyze software requirements and use cases, produce sequence diagrams as a requirements specification, decompose a complex problem into sub-problems, and start understanding and addressing software architecture design.

Ask questions on the course Slack channel or by email.

Group

Each student will be assigned to a group of five students. Please see eClass to get information about the class. You can ask the TAs if you would like to change a group. However, it may not always be possible to change your group if the other group does not want to change.

Policies

Your (submitted or un-submitted) solution to this assignment (which is not revealed to the public) remains the property of the EECS department. Do not distribute or share your code in any public media (e.g., a non-private GitHub repository) in any way, shape, or form. The department reserves the right to take necessary actions upon finding violations of this policy.

- You are highly encouraged to work in a group for this project deliverable.
- When you submit your solution, you claim that it is solely your work. Therefore, it is considered a violation of academic integrity if you copy or share any parts of your code or documentation.
- When assessing your submission, the instructor and TA may examine your doc/code, and suspicious submissions will be reported to the department/faculty if necessary. We do not tolerate academic dishonesty, so please obey this policy strictly.
- Emailing your solutions to the instructor or TAs will not be acceptable.

Submission of your work

A 2% penalty will be applied if you do not follow the following rules.

- (1) A PDF file with all the task outputs. This will be uploaded to eClass.
- (2) A link in the PDF file to a Github repo where you may have more results related to the tasks, such as UMLet output (if you feel that should also be shared).

The “system-to-be”: YorkU Conference Room Scheduler

York University’s Event Services is looking for a robust room booking platform to handle meeting and conference reservations for students, faculty, administrative staff, and external partners. The application will be developed as a GUI-based Java project. From discussions with the Event Services team, the system should meet the following:

- **Req1:** Anyone can create an account using a unique, valid email and a strong password (uppercase, lowercase, numbers, and symbols). Account types are - student, faculty, staff, and partner, with the flexibility to add new types. University accounts require verification.
- **Req2:** Only the chief event coordinator (one person) can auto-generate administrator accounts for room management.
- **Req3:** Registered users can book available rooms. Hourly rates vary: \$20 for students, \$30 for faculty, \$40 for staff, \$50 for partners.
- **Req4:** One hour’s fee is charged upfront. If the user does not check in within 30 minutes of the start time, the deposit is lost. Otherwise, it is applied to the final cost.
- **Req5:** Each room has sensors to detect occupancy and scan ID badges for verification. Data is sent to the system.
- **Req6:** Admins can add, enable, or disable rooms. Rooms can be closed temporarily for repairs or maintenance.
- **Req7:** Rooms have unique identification numbers, capacity info, and building/room location details.
- **Req8:** Users must provide a valid organization ID or student number. Bookings can be edited or canceled before the start time.
- **Req9:** Bookings may be extended before expiry if the room is available.
- **Req10:** Supports credit cards, debit cards, and institutional billing.

In Deliverable I, you are asked to analyze the raw user requirements from clients and create use cases, activity diagrams, and sequence diagrams to illustrate the requirements with more detail. After that, after that you are further asked to create a class diagram that meets the requirements. Note, no design patterns are required at Deliverable-I; we will explore the effect of design patterns on your project in Deliverable-II.

Part I: Requirements Eliciting and Modeling (50 points)

Information from the interview might not cover all the requirements; you can make your own assumptions if necessary. Please draw the diagrams carefully and present as many details as possible. A real-world developer should be able to develop the system based on these diagrams.

Task 1: Use Case Diagram (20)

Your first task is to identify the main actors in the system (hints: at least three). For each of the actors, please identify their use cases under different scenarios and draw a use case diagram for each scenario. Please also indicate the sources of your use cases (i.e., from which Reqs).

Task 2: Activity Diagram (15)

For each of your use case diagrams drawn in Task 1, please draw the activity diagram accordingly.

Task 3: Sequence Diagram (15)

For each of your use case diagrams drawn in Task 1, please draw at least one sequence diagram accordingly.

Part II: Design (20 points)

The diagrams you drew in Part I can help developers understand the basic requirements and functionalities of the 'system-to-be'. In this part, we will further explore how to design the 'system-to-be'.

Task 1: Design (Class Diagram)

Before we start the implementation task, you are asked to design the system via drawing detailed class diagrams. Please identify all the possible classes (including their attributes, methods, and relations among classes) and further draw the class diagrams of the 'system-to-be'. Note that using design patterns at the current stage is not mandatory; we will explore how we can use design patterns to improve your designs in Project Deliverable II.

Part III: Report on AI-Assistant Adaptation (30 points)

In this section, you will detail how you leveraged AI assistance throughout the requirements elicitation, modeling, and design phases of this assignment. The goal is to assess your ability to effectively use AI tools as a professional developer while demonstrating critical thinking, verification, and adaptation of the generated content.

Task1: Explain the Role of the AI Assistant

For each of the tasks in Part I and Part II, describe how you used the AI assistants (e.g., ChatGPT, Copilot, etc.). It may happen that you have not used any of the assistants, or you used those very sporadically. However, the goal for Part III is to assess your abilities to leverage such AI assistants for software design tasks. You may find the AI assistants capable of completing the tasks or fail miserably. Either way, you

cannot simply rely on the outputs of the AI assistants and need to improve the overall design, due to the various quality concerns that we often encounter with AI assistants.

Your answer should identify the specific AI tools used and explain how the AI assisted in interpreting interview information, defining system actors, and identifying use cases. Include specific examples showing how interactions with the AI influenced the structure or content of the use case, activity, sequence, or class diagrams.

Please provide the following information:

For each task, provide the following:

- (1) the AI assistant models you tried.
- (2) The prompts you tried
- (3) The output generated by each prompt
- (4) Your understanding of the output

Task 2: Arguments and Resolution

For each task in Part I and Part II, discuss whether there were instances where your understanding of the system requirements differed from the AI assistant's suggestions. Describe these disagreements and explain how they were resolved. Your answer should present the specific scenario in which the AI's logic related to a diagram component was challenged or corrected and clearly explain the reasoning process used to determine the correct design approach.

Note that we expect you to find significant problems in the AI assistant outputs. We expect to see your detailed breakdown and assessment of the problems you encountered.

Task 3: Verifying AI Responses

Explain how the accuracy and completeness of the AI assistant's responses were verified, particularly with respect to Unified Modeling Language (UML) standards. Outline the verification process used and provide an example of an AI-generated suggestion that required factual or technical verification. State the outcome of this verification and whether the suggestion was accepted, modified, or rejected.

Note that we expect to see a detailed assessment from you about your verification process.

Task 4: Application of AI-Suggested Content

Provide a concrete example of AI-suggested content that was applied directly to Part I or Part II of the assignment. Your answer should include the original prompt used to generate the content and quote the exact AI response that was applied. Clearly indicate where this content appears in the final submission and briefly explain the rationale for accepting the AI's output without significant modification.

Note that we will not be surprised to see significant modifications in your final report for the outputs of Part I and Part II than the ones produced by the AI assistant. In fact, that would be the most likely outcome.